

Git & GitHub¹

Interview Questions & Answers

100 Most Asked Questions

Beginner → Advanced



Master Git. Ace Interviews. Build Amazing Projects.

What is Git?

Git is a **Distributed Version Control System** that helps developers track changes in code, collaborate with others, and manage projects efficiently.

Why Learn Git & GitHub?

- ✓ Industry Standard
- ✓ Essential for Developers
- ✓ Used in Every Software Project
- ✓ Helps in Collaboration
- ✓ Improves Productivity

This Handbook Covers

- Git Basics
- Commands
- Branching
- Advanced Topics
- GitHub
- Real-life Scenarios
- Interview Questions
- Cheat Sheet



1. What is Git?

- Git is a distributed version control system.
- It helps track changes in files over time.
- Developed by Linus Torvalds in 2005.

Example

You can go back to any previous version of your project anytime.

Interview Answer: Git is a distributed VCS used to track changes in source code during software development.

Common Mistake:

Many people think Git and GitHub are the same. They are not!

2. Why is Git used?

- Tracks changes in code
- Allows multiple developers to work together
- Maintains history of every change
- Easy to revert to previous versions
- Supports branching and merging

Real-world Example

In a team project, if a bug occurs, we can go back to a working version of the code.

3. What is Version Control System?

A Version Control System (VCS) records changes to files over time so you can recall specific versions later.

Types of VCS

1. Local VCS
2. Centralized VCS
3. Distributed VCS
(Git is Distributed)

4. Git vs GitHub?

	Git	GitHub
Definition	Git is a tool (VCS)	GitHub is a platform
Type	Command Line Tool	Web-based Service
Where it runs?	Local Machine	Cloud (Remote)
Purpose	Track changes in code	Host Git repositories & collaboration
Internet Required?	No	Yes
Example	git commit, git push	Create repo, PR, Issues

(11). How to Install Git?

- Download from official website: git-scm.com
- Verify: `git --version`

Example (Windows)

Download → Install → Open Git Bash
→ `git --version`

(12). How to Configure Git?

```
git config --global user.name "Your Name"  
git config --global user.email "you@example.com"
```

Use Case

Sets your identity for commits.

(13). How to Initialize a Repository?

```
git init
```

Use Case

Creates a new local Git repository.



(14). How to Clone a Repository?

```
git clone <repository_url>
```

Use Case

Download a remote repo to local machine.



(15). How to Check Status?

```
git status
```

Use Case

Shows changed, staged and untracked files.



(16). How to Add Files?

```
git add <file_name>  
* or  
git add .
```

Use Case

Moves changes from working directory to staging area.



(17). How to Commit?

```
git commit -m "Your message"
```

Use Case

Saves changes permanently in local repository.



(18). How to View Commit History?

```
git log
```

Use Case

Shows all commits with details.



(19). How to Restore a File?

```
git restore <file_name>
```

Use Case

Discards changes in working directory.



(20). How to Remove a File?

```
git rm <file_name>
```

Use Case

Removes file from repo and working directory.



Branching in Git (Q21 – Q30)

21. What is a Branch?

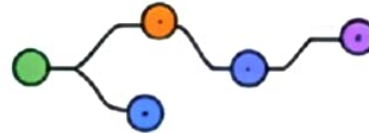
A branch is like a new line of development. It allows you to work on features independently without affecting the main code.

Example

Think of it as a copy of your project where you can experiment freely.

22. Why Branching is Important?

- Work on multiple features simultaneously
- Keep main code stable
- Easy collaboration
- Merge features after testing



23. How to Create a Branch?

```
git branch <branch_name>
```

Example

```
git branch feature-login
```

24. How to Switch to a Branch?

```
git checkout <branch_name>
```

Example

```
git checkout feature-login
```

25. How to Create & Switch to a Branch Together?

```
git checkout -b <branch_name>
```

Example

```
git checkout -b feature-login
```

26. How to List Branches?

```
git branch
```

Use Case

Lists all local branches.
* shows current branch.

27. How to Delete a Branch?

```
git branch -d <branch_name>
* force delete
git branch -D <branch_name>
```

Use Case

Deletes a branch after merging.



28. What is Merge?

Merge is used to combine changes from one branch into another.

Example

```
git merge feature-login
```

29. What is Fast Forward Merge?

If there is no divergence between branches, Git simply moves the pointer forward.

main: → A — B → C

feature: ↘ A — B — C

Fast Forward

Use Case

No extra merge commit is created.

30. What is Merge Conflict?

When Git cannot automatically merge changes, a conflict occurs.



Use Case

Manually fix conflicts and then commit.

31. Fetch vs Pull

```
git fetch    → Downloads changes
git pull    → Fetch + Merge
```

Tip

Use fetch to see changes before merging.



32. Push

```
git push origin <branch_name>
Sends your changes to remote repo.
```



33. Rebase

```
git rebase <branch_name>
Moves or combines a sequence of commits to a new base commit.
```

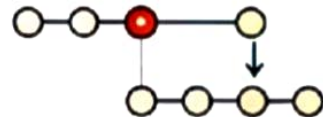
Note

Rebase keeps history clean and linear.

34. Cherry-pick

```
git cherry-pick <commit_id>
Applies specific commit from one branch to another.
```

Pick this commit



35. Stash

```
git stash    → Save changes
git stash pop → Apply stashed changes
git stash list → List all stashes
```

Use Case

When you want to switch branch but have uncommitted changes.

36. Tag

```
git tag <tag_name>
git push origin <tag_name>
```

Tip

Tags are used for release versions.



37. Reset

```
git reset --soft <commit_id> (Keep changes)
git reset --mixed <commit_id> (Unstage changes)
git reset --hard <commit_id> (Discard changes)
```

38. Revert

```
git revert <commit_id>
Creates new commit to undo changes (safe for shared branches).
```

Note

Revert is safe, Reset is not (for public branches).

39. Clean

```
git clean -n    → Dry run (preview)
git clean -f    → Remove untracked files
git clean -fd   → Remove untracked files & dirs
```



40. Diff

```
git diff        → Unstaged changes
git diff --staged → Staged changes
git diff <c1> <c2> → Between two commits
```

Tip

Diff helps you see exact changes.

41. What is Fork?

A copy of repository in your GitHub account.

Use Case

To contribute to someone else's project.



42. What is Clone?

Download a repository to your local machine.

Command

`git clone <repo_url>`



43. What is Pull Request (PR)?

Request to merge your changes into the original repository.

Use Case

Used in collaboration to review & merge code.



44. What is Code Review?

Reviewing code to find bugs, improve quality and maintain standards.

Tip

Good code review improves code quality & team knowledge.



45. What are Issues?

Used to track bugs, tasks, or feature requests.

Example

Bug report, Feature request, Documentation



46. What are Discussions?

Used for general discussions, Q&A and idea sharing.

Tip

Great for open-source communities.



47. What are Releases?

Used to package and publish project versions.

Example

v1.0.0, v2.1.0 (with release notes)



48. What is a Star?

Star a repo to show interest.

Tip

More stars = More visibility.



49. What is Watch?

Get notifications about activities in a repository.

Tip

Useful to stay updated.



50. What is GitHub Actions?

CI/CD tool built into GitHub.

Example

Automate tests, build, deploy etc.



51. What is Gist?

Share code snippets.

Use Case

Share small pieces of code quickly.



52. What is README.md?

First file people see in your repo. Contains project info.

Tip

Good README = Good first impression.



53. What is .gitignore?

Tells Git which files/folders to ignore.

Example

node_modules/, .env, *.log



54. What is License?

Defines how others can use your project.

Tip

MIT, Apache 2.0 are popular licenses.



55. What is GitHub Pages?

Host static websites directly from repo.

Use Case

Portfolio, docs, projects.



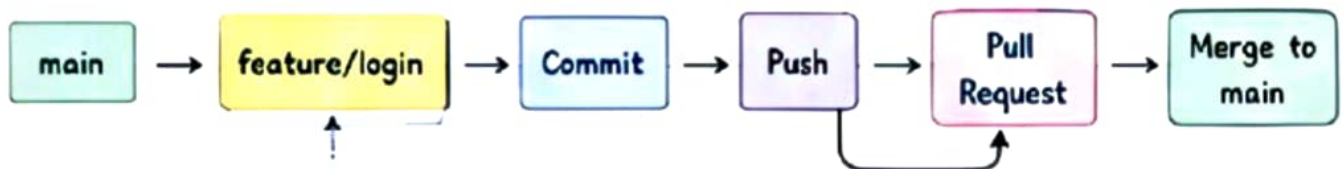
Team Collaboration & Workflow (Q56 - Q70)

7

56. Typical Team Workflow



57. Feature Branch Workflow



58. Git Flow (Simplified)

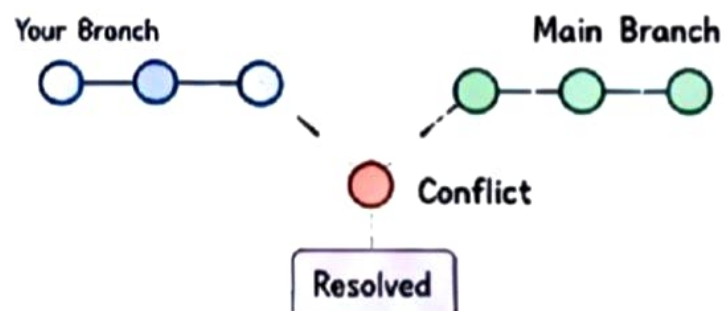


59. Best Practices

- ✓ Commit small, meaningful changes.
- ✓ Write clear commit messages.
- ✓ Pull latest changes before starting work.
- ✓ Use meaningful branch names.
- ✓ Review code before merging.
- ✓ Keep main branch clean.
- ✓ Resolve conflicts locally.
- ✓ Don't commit sensitive data.
- ✓ Use .gitignore properly.
- ✓ Communicate with team.

60. Tips for Conflict Resolution

- ✓ Pull latest changes.
- ✓ Check conflicting files.
- ✓ Edit and fix conflicts.
- ✓ Stage resolved files.
- ✓ Commit the changes.



Remember: Good workflow = Happy Team + Quality Code + Successful Projects

71. Q: Accidentally deleted a commit?

Solution: `git reflog` → find commit id → `git reset --hard <id>`



72. Q: Wrong branch me commit kar diya?

Solution: `git stash` → switch branch → `git stash pop`



73. Q: Last commit undo karna hai (keep changes)?

Solution: `git reset --soft HEAD-1`



74. Q: Last commit undo karna hai (discard changes)?

Solution: `git reset --hard HEAD-1`



75. Q: Specific commit copy karna hai?

Solution: `git cherry-pick <commit_id>`



76. Q: Merge conflict aa gaya, kya kare?

Solution:

1. `git status`
2. Conflicting files edit karo
3. `git add <file>`
4. `git commit`



77. Q: Local changes remote pe force push karna hai?

Solution: `git push origin <branch> --force`



78. Q: Untracked files remove karna hai?

Solution: `git clean -fd`



79. Q: Stash kiya hua changes wapas lana hai?

Solution: `git stash pop`



80. Q: Kya difference hai reset aur revert me?

Solution:

- Reset history change karta hai (dangerous for public branch).
- Revert new commit banata hai (safe for public branch).



81. Difference between Reset and Revert?

Answer:

- Reset moves HEAD to a previous state and can change history.
- Revert creates a new commit to undo changes without affecting history.

Interview Tip

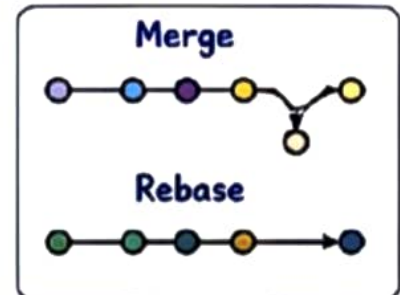
Use Reset for local changes, Revert for shared (public) changes.



82. Difference between Merge and Rebase?

Answer:

- Merge combines branches and creates a merge commit.
- Rebase rewrites history by applying commits on top of another branch.



83. What is HEAD in Git?

Answer: HEAD is a pointer that refers to the current branch and the latest commit in that branch.

Think: HEAD = You are here



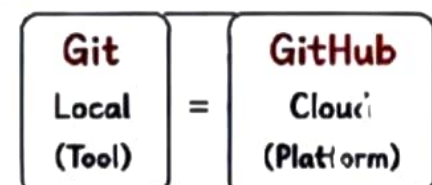
84. What is .gitignore file?

Answer: It tells Git which files or folders to ignore (not to track). Example: node_modules/, .env, .log

```
.gitignore
node_modules/
.env
.log
```

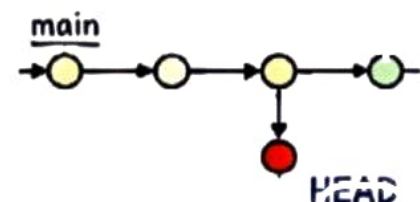
85. What is the difference between Git and GitHub?

Answer: Git is a Version Control System (VCS). GitHub is a hosting platform for Git repositories with additional features like issues, PR, Actions, etc.



86. What is a Detached HEAD?

Answer: When HEAD points to a specific commit instead of a branch. You can explore the code but new commits won't belong to any branch.



87. What is the purpose of Tag?

Answer: Tags are used to mark important points in history (like releases).



88. How to squash multiple commits into one?

Answer: Use interactive rebase.

`git rebase -i HEAD~n`

Pick 'squash' for extra commits.

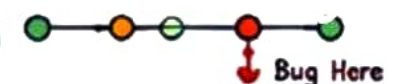
89. What is a bare repository?

Answer: A repository without a working directory. It only contains the .git folder. Used for central repo.



90. What is Git Bisect?

Answer: It helps to find the commit that introduced a bug using binary search.



Git Cheat Sheet ' (Important Commands)

11

Setup

Command	Description
git config --global user.name "Your Name"	Set username
git config --global user.email "your@email.com"	Set email
git config --list	Show configurations

Repository

Command	Description
git init	Initialize new repository
git clone <url>	Clone a repository
git status	Show current status
git log	Show commit history
git log --oneline --graph --all	Graphical log view

Basic Workflow

Command	Description
git add .	Stage all changes
git add <file>	Stage a specific file
git commit -m "message"	Commit changes
git restore <file>	Discard changes in working dir
git rm <file>	Remove file and stage
git commit -am "message"	Add & commit tracked changes

Branching

Command	Description
git branch	List branches
git branch <name>	Create branch
git checkout <name>	Switch branch
git checkout -b <name>	Create & switch branch
git merge <branch>	Merge branch
git branch -d <name>	Delete branch

Remote

Command	Description
git remote -v	Show remotes
git remote add origin <url>	Add remote
git fetch origin	Fetch changes
git pull origin <branch>	Fetch & merge
git push origin <branch>	Push changes
git push -u origin <branch>	Push & set upstream

Advanced

Command	Description
git diff	Show changes
git diff --staged	Show staged changes
git stash	Stash changes
git stash pop	Apply stashed changes
git reset --soft HEAD-1	Undo commit (keep changes)
git reset --hard HEAD-1	Undo commit (discard changes)
git revert <commit_id>	Undo commit (safe)
git rebase <branch>	Reapply commits
git cherry-pick <id>	Apply specific commit
git tag	List tags
git tag <tag_name>	Create tag
git clean -fd	Remove untracked files & dirs

✗ Common Errors & Fixes

- ✗ error: failed to push
→ Pull latest changes and try again.
- ✗ Merge conflict
→ Resolve conflict, add, and commit.
- ✗ Detached HEAD
→ Checkout a branch to fix.
- ✗ Untracked files would be overwritten
→ Stash or commit your changes.



Real Project Git Workflow (From Start to Deployment)

10

- 1**  **Clone the Repository**
Get the remote repository on your local machine.
`git clone <repo_url>`
- 2**  **Create a New Branch**
Always work on a new branch.
`git checkout -b feature/login`
- 3**  **Make Changes**
Write code and make necessary changes.

- 4**  **Stage Changes**
Add changes to staging area.
`git add .`
- 5**  **Commit Changes**
Save changes with a meaningful commit message.
`git commit -m "Add login feature"`
- 6**  **Push to Remote**
Upload your branch to GitHub.
`git push origin feature/login`
- 7**  **Create Pull Request**
Raise PR for review and feedback.
`Pull Request` 
- 8**  **Code Review**
Reviewers check your code and suggest changes.

- 9**  **Merge the PR**
After approval, PR is merged into main branch.

- 10**  **Pull Latest & Deploy**
Pull latest code and deploy to production.
`git pull origin main`



Good workflow = Clean Code + Smooth Collaboration
+ Happy Team + Successful Project



Top 20 Must-Remember Commands

1. git status
2. git add .
3. git commit -m "msg"
4. git log --oneline
5. git diff
6. git branch
7. git checkout <branch>
8. git checkout -b <branch>
9. git merge <branch>
10. git pull origin <branch>
11. git push origin <branch>
12. git stash
13. git stash pop
14. git reset --soft HEAD~1
15. git reset --hard HEAD~1
16. git revert <id>
17. git cherry-pick <id>
18. git tag <name>
19. git remote -v
20. git clean -fd

Interview Do's & Don'ts

✓ Do's

- ✓ Explain with examples
- ✓ Understand the use case
- ✓ Be honest if you don't know
- ✓ Stay calm & think out loud

✗ Don'ts

- ✗ Don't memorize blindly
- ✗ Don't panic on tough questions
- ✗ Don't give wrong answers
- ✗ Don't ignore basics

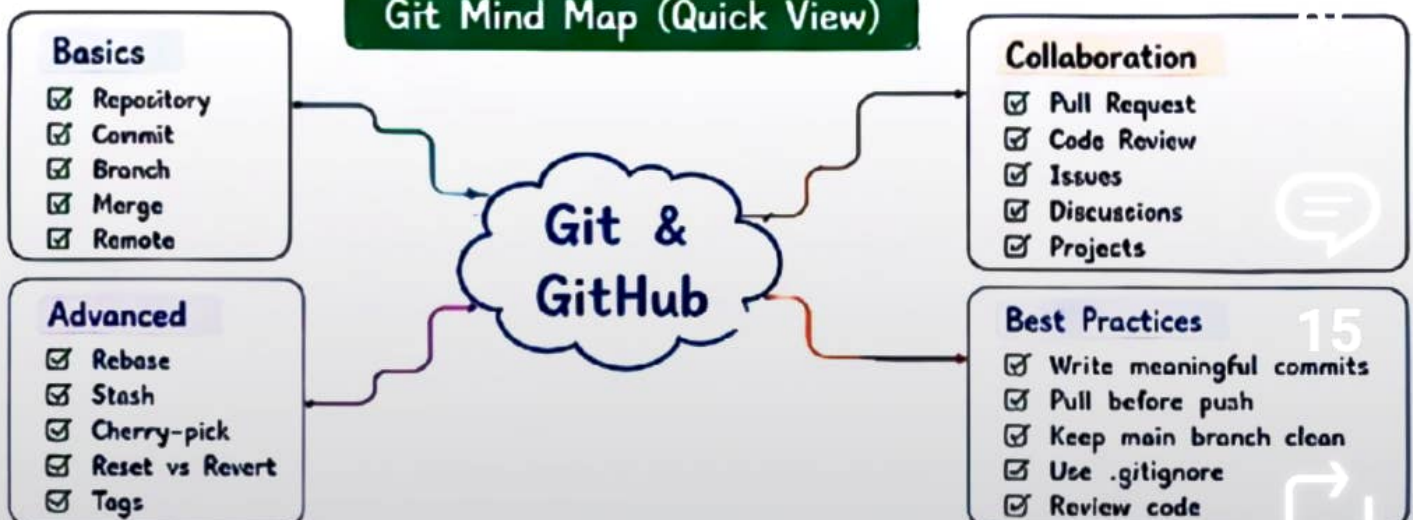
Learning Roadmap



Recommended Resources

- ☐ Official Git Documentation
🔗 git-scm.com/doc
- 📖 Pro Git Book (Free)
🔗 <https://git-scm.com/book/en/v2>
- 📺 YouTube: FreeCodeCamp
- 📖 GitHub Docs
🔗 docs.github.com
- 👤 Practice: GitHub
~(Create repos, contribute)

Git Mind Map (Quick View)



"Every Commit Counts. Keep Learning, Keep Growing!"